

Data Analysis with Python 2024–2025

Parte 3: NumPy (Parte 2)

Conteúdo Teórico

Tradução e Adaptação: The University of Helsinki MOOC Center

Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024–2025*, oferecido pela **The University of Helsinki MOOC Center**.

O que você vai aprender nesta página

- Aprender a comparar, criar máscaras, indexar e ordenar arrays.
- Entender como usar o NumPy para multiplicação de matrizes e para resolver sistemas de equações lineares.

Conteúdo

1 Comparações e máscaras

Assim como o NumPy permite operações aritméticas elemento a elemento entre arrays, como a soma de dois arrays, também é possível comparar dois arrays elemento a elemento.

```
import numpy as np

a = np.array([1,3,4])
b = np.array([2,2,7])
c = a < b
print(c)
print(c.all()) # todas as comparacoes foram True?
print(c.any()) # alguma comparacao foi True?
print(np.sum(c))
print(a > 0)
```

A comparação produz um array booleano, por exemplo `[True, False, True]`. O método `all()` verifica se todos os elementos são verdadeiros, enquanto `any()` verifica se pelo menos um é verdadeiro. Como `True` equivale a 1 e `False` a 0 em operações numéricas, `np.sum(c)` conta quantas comparações foram verdadeiras.

As regras de *broadcasting* também se aplicam às comparações.

1.1 Exemplo com dados meteorológicos

Para experimentar essas operações com dados reais, o material carrega estatísticas meteorológicas da estação Kumpula, em Helsinque. São usadas as temperaturas médias diárias do ano de 2017. O Pandas é utilizado apenas para carregar o arquivo de dados.

```
import pandas as pd

a = pd.read_csv("https://raw.githubusercontent.com/csmastersUH/
data_analysis_with_python_2020/master/kumpula-weather-2017.csv")["Air
temperature (degC)"].values

print("Number of days with the temperature below zero", np.sum(a < 0))
print(np.sum((0 < a) & (a < 10)))
```

No conjunto de dados apresentado, há 49 dias abaixo de zero. Para arrays booleanos, usamos os operadores elemento a elemento `&`, `|` e `~`, correspondentes, respectivamente, a "e", "ou" e "não". Os parênteses das comparações são importantes porque a precedência de `&` é maior do que a de `<`.

Uma máscara booleana também pode selecionar um subconjunto do array:

```
c = a > 0
print(c[:10])
print(a[c])
```

Essa operação é chamada de *masking* (mascaramento). A máscara também pode ser usada para atribuir valores:

```
a[~c] = 0
print(a)
```

No exemplo, todas as temperaturas negativas são substituídas por zero.

2 Indexação avançada (fancy indexing)

Com indexação simples podemos obter um único elemento de um array. Para obter vários elementos, que não precisam ser contíguos, podemos fornecer uma lista de índices.

```

np.random.seed(0)
a = np.random.randint(0, 20, 20)
a2 = np.array([a[2], a[5], a[7]])
print(a)
print(a2)

idx = [2, 5, 7]
print(a[idx])
print(a[[2, 5, 7]])

a[idx] = -1
print(a)

```

A indexação avançada também funciona em arrays de dimensões maiores. Em um array bidimensional, é possível fornecer dois arrays de índices, um para as linhas e outro para as colunas. O *shape* do resultado é determinado pelo *shape* dos arrays de índices, e não pelo *shape* do array original.

O *broadcasting* pode ser combinado com a indexação avançada para evitar repetições:

```

b = np.arange(16).reshape(4, 4)
row = np.array([0, 2])
col = np.array([1, 3])

row2 = row.reshape(2, 1)
col2 = col.reshape(1, 2)

print(b[row2, col2])
print(b[:, [0, 2]])

```

3 Ordenação de arrays

Há duas formas principais de ordenar arrays unidimensionais. `np.sort(a)` retorna um novo array ordenado e não modifica o argumento. Já `a.sort()` modifica o próprio array.

```

a = np.array([2, 1, 4, 3, 5])
print(np.sort(a))
print(a)
a.sort()
print(a)

```

Em arrays multidimensionais, o parâmetro `axis` define se queremos ordenar cada coluna ou cada linha:

```

b = np.random.randint(0, 10, (4, 4))
print(np.sort(b, axis=0)) # cada coluna
print(np.sort(b, axis=1)) # cada linha

```

Cada linha ou coluna é ordenada independentemente.

Uma operação relacionada é `np.argsort`. Em vez de ordenar os valores, ela retorna os índices que colocariam os valores em ordem.

```

a = np.array([23, 12, 47, 35, 59])
idx = np.argsort(a)
print("Array a:", a)
print("Indices:", idx)
print(a[idx])

```

No exemplo, os índices indicam a ordem dos elementos: o menor valor está na posição 1, o segundo menor na posição 0, o terceiro na posição 3, e assim por diante.

4 Operações matriciais

O operador `*` do NumPy corresponde ao produto elemento a elemento. Muitas vezes, porém, queremos a multiplicação matricial.

Se A tem *shape* $n \times k$ e B tem *shape* $k \times m$, o produto matricial $C = AB$ tem *shape* $n \times m$, com

$$C_{ij} = \sum_{h=1}^k a_{ih}b_{hj}$$

No NumPy, podemos usar `np.matmul` ou o operador `@`.

```
np.random.seed(0)
a = np.random.randint(0, 10, (3, 2))
b = np.random.randint(0, 10, (2, 4))
c = np.matmul(a, b)
print(c)
print(a @ b)
```

Para o produto matricial ser definido, as dimensões internas das matrizes precisam ser compatíveis. Assim, uma tentativa como `a @ a`, quando as dimensões não forem compatíveis, produzirá um erro.

4.1 Matriz inversa

Para uma matriz invertível, podemos calcular a inversa com `np.linalg.inv`. Se A^{-1} é a inversa de A , os produtos AA^{-1} e $A^{-1}A$ devem ser aproximadamente a matriz identidade. Pequenos valores numéricos próximos de zero podem aparecer devido à aritmética de ponto flutuante.

```
s = np.array([[1, 6, 7], [7, 8, 1], [5, 9, 8]])
s_inv = np.linalg.inv(s)
print("Original matrix:\n", s)
print("Inverted matrix:\n", s_inv)
print("Matrix product:\n", s @ s_inv)
print("Matrix product the other way:\n", s_inv @ s)
```

Também podemos transformar um vetor com uma matriz e depois aplicar a inversa para recuperar o vetor original.

5 Resolução de sistemas de equações lineares

O NumPy fornece `np.linalg.solve` para resolver sistemas de equações lineares. A função recebe a matriz de coeficientes e o vetor dos termos constantes.

Um sistema geral pode ser escrito como

$$Ax = b$$

onde A contém os coeficientes, x é o vetor de incógnitas e b contém os termos do lado direito. No formato descrito no material, A possui *shape* (n, m) , x possui *shape* $(m, 1)$ e b possui *shape* $(n, 1)$.

Créditos: Este conteúdo foi originalmente criado pela *The University of Helsinki MOOC Center* para o curso *Data Analysis with Python*.